

```

1 import java.util.Arrays;
2 import java.util.PriorityQueue;
3
4 public class TSPBranchAndBound {
5
6     static int N; // Number of cities
7     static int[][] costMatrix; // The cost matrix
8
9     // Node class to store a partial tour, its cost, and its current bound
10    static class Node implements Comparable<Node> {
11        int[] path; // Current path of the cities visited
12        int level; // Level of the node in the tree (number of cities visited)
13        int bound; // Lower bound of the node
14        int cost; // Current cost of the path
15
16        // Constructor
17        Node(int[] path, int level, int cost, int bound) {
18            this.path = path.clone();
19            this.level = level;
20            this.cost = cost;
21            this.bound = bound;
22        }
23
24        // Compare nodes based on their bound (for priority queue)
25        public int compareTo(Node other) {
26            return this.bound - other.bound;
27        }
28    }
29
30    // Function to calculate the lower bound of a node
31    static int calculateBound(Node node) {
32        int bound = node.cost;
33        boolean[] visited = new boolean[N];
34
35        // Mark visited cities
36        for (int i = 0; i <= node.level; i++) {
37            visited[node.path[i]] = true;
38        }
39
40        // For each unvisited city, add the minimum cost edge
41        for (int i = 0; i < N; i++) {
42            if (!visited[i]) {
43                int minCost = Integer.MAX_VALUE;
44                for (int j = 0; j < N; j++) {
45                    if (!visited[j] && i != j) {
46                        minCost = Math.min(minCost, costMatrix[i][j]);
47                    }
48                }
49                bound += minCost;
50            }
51        }
52        return bound;
53    }
54
55    // Function to solve TSP using Branch and Bound
56    static int solveTSP() {
57        PriorityQueue<Node> pq = new PriorityQueue<>();
58        int[] initialPath = new int[N + 1]; // Store cities in path
59
60        // Start from city 0
61        initialPath[0] = 0;
62
63        // Create the root node
64        Node root = new Node(initialPath, 0, 0, calculateBound(new Node(initialPath, 0, 0, 0)));
65
66        // Add root node to the priority queue
67        pq.add(root);
68
69        // Track the minimum cost and path
70        int minCost = Integer.MAX_VALUE;

```

```

70 // minCost = Integer.MAX_VALUE;
71 int[] bestPath = null;
72
73 // Branch and Bound Loop
74 while (!pq.isEmpty()) {
75     Node current = pq.poll();
76
77     // Prune the node if its bound exceeds the minimum cost found so far
78     if (current.bound < minCost) {
79         // If we've visited all cities and are back at the start, check if it's a better solution
80         if (current.level == N - 1) {
81             int lastCost = costMatrix[current.path[current.level]][current.path[0]];
82             if (lastCost != Integer.MAX_VALUE && current.cost + lastCost < minCost) {
83                 minCost = current.cost + lastCost;
84                 bestPath = current.path.clone();
85                 bestPath[N] = bestPath[0]; // Complete the cycle
86             }
87         } else {
88             // Branch out to other cities
89             for (int i = 1; i < N; i++) {
90                 if (!isCityVisited(current.path, current.level, i)) {
91                     int[] newPath = current.path.clone();
92                     newPath[current.level + 1] = i;
93                     int newCost = current.cost + costMatrix[current.path[current.level]][i];
94                     Node childNode = new Node(newPath, current.level + 1, newCost, 0);
95                     childNode.bound = calculateBound(childNode);
96
97                     // If the bound is better than the current minCost, add it to the PQ
98                     if (childNode.bound < minCost) {
99                         pq.add(childNode);
100                     }
101                 }
102             }
103         }
104     }
105 }
106
107 // Print the best path found
108 if (bestPath != null) {
109     System.out.println("Best Path: " + Arrays.toString(bestPath));
110 }
111
112 return minCost;
113 }
114
115 // Helper function to check if a city has been visited in the current path
116 static boolean isCityVisited(int[] path, int level, int city) {
117     for (int i = 0; i <= level; i++) {
118         if (path[i] == city) {
119             return true;
120         }
121     }
122     return false;
123 }
124
125 public static void main(String[] args) {
126     N = 4; // Number of cities
127     costMatrix = new int[][]{
128         {Integer.MAX_VALUE, 10, 15, 20},
129         {10, Integer.MAX_VALUE, 35, 25},
130         {15, 35, Integer.MAX_VALUE, 30},
131         {20, 25, 30, Integer.MAX_VALUE}
132     };
133
134     int minCost = solveTSP();
135     System.out.println("Minimum cost of TSP: " + minCost);
136 }
137 }
138
139
140 /*

```

```
141 Output:
142
143 Best Path: [0, 1, 3, 2, 0]
144 Minimum cost of TSP: 80
145
146 * */
```